

TP3 - Programmation Orientée Objet en TypeScript

Dans ce TP, nous allons créer un système d'API de livres basé sur Express.js. Cela comprend le back-end, avec alimentation d'une base de données qui permettra de retourner des données, et le front-end qui utilisera ces données pour alimenter une application.

Faire l'installation des modules npm et de Typescript en partant des fichiers en annexe (*package.json* et *tsconfig.json*).

Exercice 1 :

Créer le dossier *src/* à la racine du projet. Celui-ci contiendra l'ensemble de notre code source.

Créer à l'intérieur de ce dernier un dossier *server/*, y intégrer un fichier *server.ts*.

Initialiser une app avec Express. Celle-ci tournera sur le port 3000 et affichera un message au lancement du serveur avec la fonction `listen()`.

Le code est disponible en annexe.

Exercice 1.1 :

Quand on envoie une requête POST à un serveur Express (par exemple via `fetch()` ou un formulaire HTML), les données ne sont pas automatiquement lisibles : elles arrivent sous forme de texte brut.

Pour qu'Express puisse les convertir en objet JavaScript accessible via `req.body`, il faut activer deux middlewares :

```
app.use(express.json());  
app.use(express.urlencoded({ extended: true }));
```

- `express.json()` : permet à Express de lire et décoder les requêtes contenant du JSON (souvent envoyées depuis `fetch()` ou une API).
- `express.urlencoded()` : permet de décoder les formulaires HTML classiques envoyés avec le format `application/x-www-form-urlencoded`.

Le paramètre `extended: true` permet de gérer correctement les objets et tableaux imbriqués dans les données du formulaire.

Sans ces deux lignes, req.body serait vide ou undefined, car Express ne saurait pas comment interpréter le contenu de la requête.

Exercice 2 :

Créer une base de données MySQL avec la structure suivante depuis phpMyAdmin :

- id - *int(11)* - AI
- title - *varchar(255)*
- author - *varchar(255)*
- pageCount - *int(11)*
- year - *int(11)*

Exercice 2.1 :

Créer un dossier *db/* dans le dossier *server/* avec un fichier nommé *connection.ts*. Celui-ci aura pour but de créer la connexion avec la base de données.

```
export const connection = mysql.createConnection({
  host: "localhost",
  user: "root",
  password: "",
  database: "",
});
```

Afficher un message contenant le résultat de la tentative de connexion.

```
connection.connect((error) => {});
```

Exercice 3 :

Pour remplir notre base de données, nous allons nous servir d'un service que nous allons définir dans notre application Express.

Dans le dossier *server/*, créer un dossier *routes/*. Celui-ci contiendra un dossier nommé *web/* et à l'intérieur un fichier *book.form.routes.ts*.

Ce fichier ajoutera la route */add-book* sur l'application. La route possède 2 méthodes.

- *get()* : permet de définir ce que l'utilisateur trouvera sur la route
- *post()* : le résultat du post de l'utilisateur

```
import { Router, Request, Response } from "express";
import { connection } from "../../db/connection.js";

const router = Router();

router.get("/add-book", (req: Request, res: Response) => {
  res.send();
});

router.post("/add-book", (req: Request, res: Response) => {});
```

Ajouter ces deux méthodes à la route /add-book

Exercice 3.1 :

Une fois la route créée, nous allons la relier à notre application.
Il suffit d'importer la route dans notre serveur, et de lui lier un chemin.

```
import bookFormRoutes from "../routes/web/book.form.routes.js";
app.use("/", bookFormRoutes);
```

Tenter de rentrer des données dans la base à l'aide de la route /add-book.

Exercice 4 :

Une fois la possibilité de mettre des livres dans la base mis en place, nous allons créer une API permettant de communiquer ces mêmes données à un autre serveur.

Créer un dossier */api* dans le dossier */routes* avec un fichier book.api.routes.ts.

```
const router = Router();

router.get("/books", (req: Request, res: Response) => {
  let sql = "SELECT * FROM books WHERE 1=1";
  const values: any[] = [];
  ... // code à compléter

  connection.query(sql, values, (error, rows) => {
    return res.json(rows);
  });
});
```

Dans le cas présent, on va construire la requête SQL en fonction des données récupérées dans l'URL. Les paramètres donnés sont accessibles via `req.query` (le paramètre d'URL `year` sera stocké dans `req.query['year']`).

Ajouter la route dans le `server.ts` en faisant précéder toutes les routes par `/api`.

```
app.use("/api", bookApiRoutes);
```

Exercice 4.1 :

Ajouter les paramètres suivant que pourra prendre l'API :

- `year` - *retourne les livres correspondants à l'année*
- `minPages` - *nombre de pages minimum que doit contenir le livre*
- `maxPages` - *nombre de pages maximum que doit contenir le livre*
- `limit` - *la limite de livres à récupérer*

Une fois ajoutés, tester plusieurs cas d'appel API retournant des listes différentes.

Exercice 5 :

Une fois le back-end terminé, nous allons construire un autre serveur qui sera chargé de récupérer les données transmises par l'API.

Créer un dossier `/client` dans `/src` avec un fichier `index.ts`.

```
async function getMessage() {  
  try {  
    const response = await fetch("");  
    const data = await response.json();  
  } catch (error) {}  
}  
getMessage();
```

Exercice 5.1 :

Avec un appel API directement sur le serveur Express, une erreur apparaît :

```
(index):1 Access to fetch at 'http://localhost:3000/api/books' from origin  
'http://localhost:5000' has been blocked by CORS policy: No  
'Access-Control-Allow-Origin' header is present on the requested resource.
```

Le CORS (Cross-Origin Resource Sharing) empêche par défaut un navigateur d'accéder à des ressources provenant d'un autre domaine ou port. Dans notre cas, le serveur Express (port 3000) doit explicitement autoriser le client (port 5000) à effectuer des requêtes via le middleware cors().

```
app.use(cors({
  origin: "http://localhost:5000"
}));
```

Cela ajoute automatiquement les bons en-têtes HTTP (Access-Control-Allow-Origin) dans les réponses envoyées par le serveur.

Nous avons maintenant une API complète nous permettant d'accéder à une base de données de livres.

Exercice 6 :

Créer la classe `Book` dans le dossier `/client/models` avec les attributs privés suivants :

- `title: string`
- `author: string`
- `year: number`
- `pageCount: number`

Le constructeur reçoit ces 4 paramètres dans l'ordre indiqué.

Ajoutez une méthode publique `toString: string` qui renvoie une chaîne lisible regroupant toutes les informations d'une instance.

"The Pragmatic Programmer" (1999) by Andrew Hunt - 352 pages

Exercice 6.1 :

Instancier des objets `Book` avec les données récupérées.

Ajouter les méthodes publiques `getReadingTime: number` et `isClassic: boolean` sur la classe `Book`.

- `getReadingTime` : permet de donner un temps de lecture approximatif. La méthode prend un paramètre `pagesPerHour` (à 50 par défaut) et retourne le temps que prendra la lecture du livre.
- `isClassic` : retourne vrai si le livre a plus de 50 ans.

Exercice 6.2 :

Créer une feuille de style permettant de mettre en forme les données récupérées. Vous pouvez ajouter des données supplémentaires dans la base pour étoffer celle-ci (court résumé, note /5, genre, etc.).

Annexes :

package.json

```
{
  "name": "tp",
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "build": "npx tsc",
    "watch": "npx tsc --watch",
    "start:server": "node dist/server/server.js",
    "dev:server": "nodemon dist/server/server.js",
    "start:client": "npx serve . -l 5000"
  },
  "dependencies": {
    "body-parser": "^2.2.0",
    "cors": "^2.8.5",
    "express": "^4.19.0",
    "mysql": "^2.18.1",
    "serve": "^14.2.5"
  },
  "devDependencies": {
    "@types/cors": "^2.8.19",
    "@types/express": "^5.0.3",
    "@types/mysql": "^2.15.27",
    "nodemon": "^3.1.10",
    "ts-node": "^10.9.2",
    "typescript": "^5.6.0"
  }
}
```

tsconfig.json

```
{
  "compilerOptions": {
    "rootDir": "./src",
    "outDir": "./dist",

    "module": "ESNext",
    "moduleResolution": "node",
    "target": "ES2022",
    "lib": ["ES2022", "DOM", "DOM.Iterable"],
    "types": ["node"],

    "sourceMap": true,
    "declaration": false,
    "declarationMap": false,

    "strict": true,
    "noImplicitOverride": true,
    "noImplicitReturns": true,
    "noPropertyAccessFromIndexSignature": true,
    "useUnknownInCatchVariables": true,
    "noUncheckedIndexedAccess": true,
    "exactOptionalPropertyTypes": true,

    "verbatimModuleSyntax": false,
    "isolatedModules": false,
    "moduleDetection": "force",
    "noUncheckedSideEffectImports": true,
    "allowSyntheticDefaultImports": true,

    "skipLibCheck": true,
    "forceConsistentCasingInFileNames": true
  },
  "include": ["src"]
}
```

server.ts

```
import express from "express";
import cors from "cors";
import bodyParser from "body-parser";

const app = express();
const port = 3000;

// Middlewares
app.use(bodyParser.urlencoded({ extended: true }));
app.use(bodyParser.json());

// Page d'accueil
app.get("/", (req, res) => {
  res.send(`Homepage serveur`);
});

// Lancement du serveur
app.listen(port, () => {
  console.log(`✅ Serveur lancé sur http://localhost:${port}`);
});
```